

ELECTIVA V · INGENIERÍA DE SOFTWARE E IA

AgenteApis

Generación automática de APIs REST mediante sistemas multiagente

Sistema basado en **FastAPI** y **LangGraph** que transforma requerimientos en lenguaje natural en código backend funcional, eliminando la escritura manual de APIs CRUD.

El coste oculto del desarrollo backend manual

Desarrollo tradicional

- Tareas repetitivas y propensas a errores
- Inconsistencias entre diseño y código
- Tiempo elevado en boilerplate CRUD
- Alta fricción en proyectos iniciales

La solución: AgenteApis

Automatizar la generación del backend inicial mediante un sistema multiagente de IA, reduciendo el tiempo de arranque y garantizando coherencia entre el diseño y el código producido.

- El desarrollador describe el dominio; los agentes generan la API completa.

¿Qué es AgenteApis?

Pipeline de IA que transforma una descripción en lenguaje natural en una API REST funcional con FastAPI, pasando por análisis, diseño, validación y generación de código.



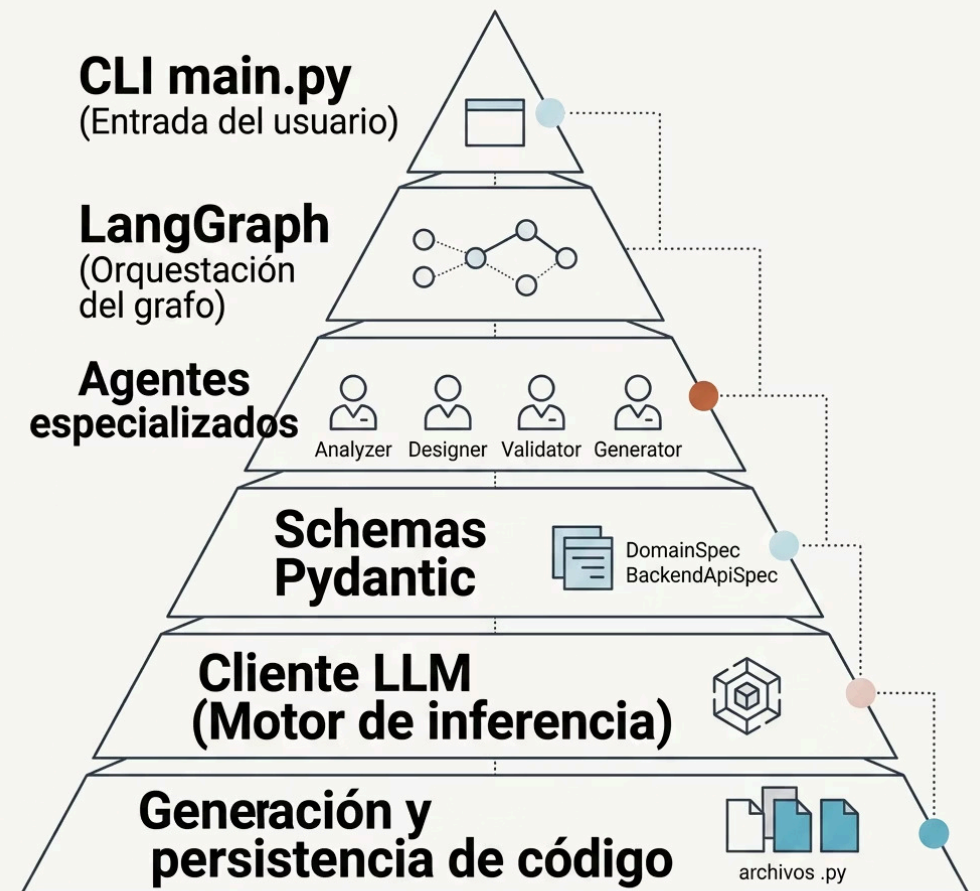
Cada etapa es ejecutada por un agente especializado que produce artefactos estructurados, garantizando trazabilidad y consistencia en todo el proceso.

Campo	Descripción
<code>user_requirement</code>	Texto que escribe el usuario en consola
<code>domain_spec</code>	Salida del analizador (dict serializado de <code>DomainSpec</code>)
<code>backend_spec</code>	Salida del diseñador (<code>BackendApiSpec</code>)
<code>backend_spec_ok</code>	Si la especificación pasó validación
<code>backend_spec_errors</code>	Lista de errores de consistencia
<code>generated_code</code>	Código Python devuelto por el generador
<code>validation_ok</code>	Si el código es aceptable
<code>validation_error</code>	Mensaje si falla validación

Capas del sistema

AgenteApis se estructura en capas claramente diferenciadas, desde la interfaz de línea de comandos hasta la persistencia del código generado, pasando por la orquestación agentic con LangGraph y el cliente LLM.

- **CLI (main.py):** punto de entrada del usuario
- **LangGraph:** orquestación del grafo de agentes
- **Agentes especializados:** roles independientes y composables
- **Schemas Pydantic:** DomainSpec y BackendApiSpec
- **Cliente LLM:** motor de inferencia de los agentes
- **Generación y persistencia de código:** escritura de archivos .py



Responsabilidades por agente



Analyzer

Interpreta el dominio del usuario y extrae entidades, relaciones y reglas de negocio.



Designer

Diseña la API: recursos, endpoints, métodos HTTP y esquemas de datos.



Validator

Verifica la consistencia de la especificación antes de la generación.



Generator

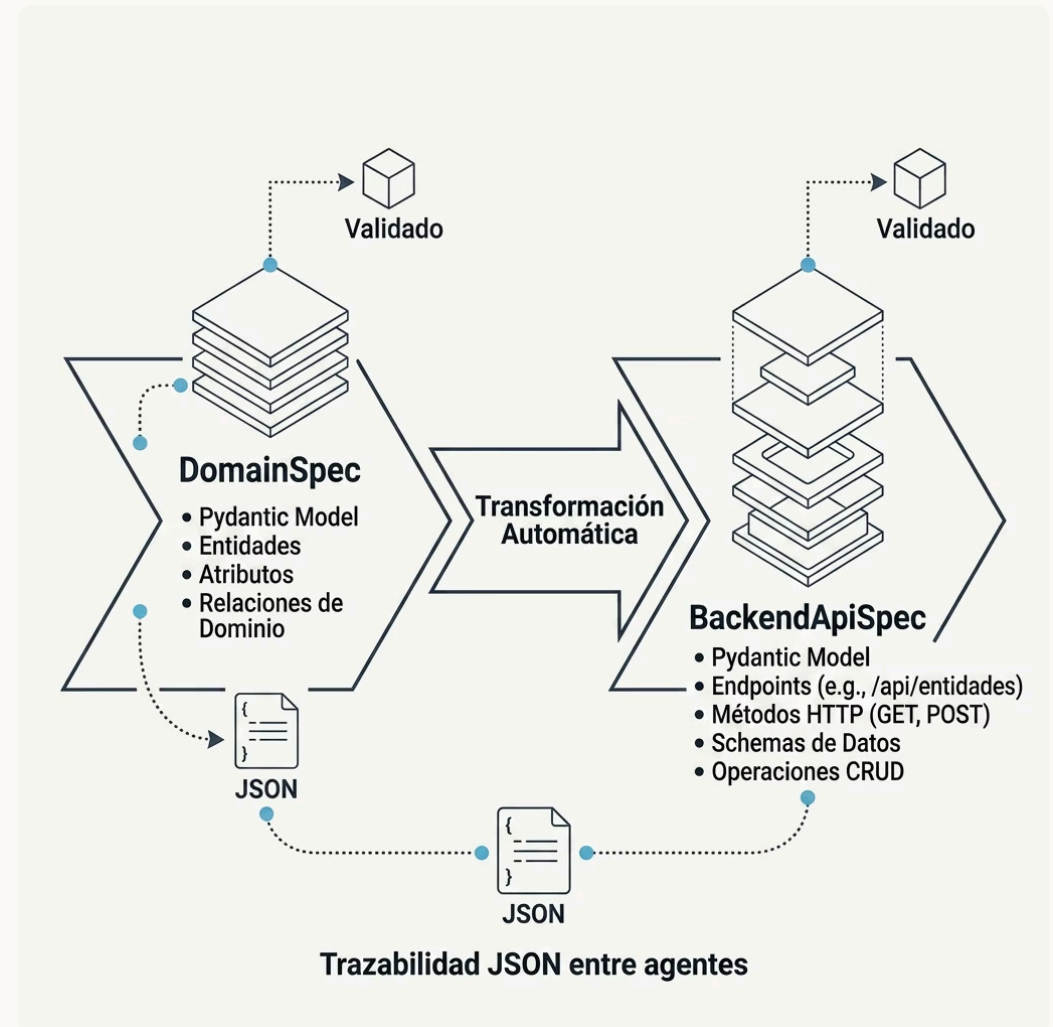
Produce el código FastAPI: modelos, rutas, CRUD y documentación Swagger.

DomainSpec y BackendApiSpec: trazabilidad estructurada

Los agentes no intercambian texto libre, sino **objetos Pydantic validados** que garantizan coherencia entre etapas:

- **DomainSpec:** entidades, atributos y relaciones del dominio
- **BackendApiSpec:** endpoints, métodos, schemas y operaciones CRUD

📄 El JSON intermedio permite auditar y depurar cada decisión del sistema.



📄 Insertar aquí captura de ejemplo JSON generado.

FastAPI listo para ejecutar

El agente Generator produce código Python completo y funcional:

- Instancia `app = FastAPI()` con metadata
- Modelos Pydantic generados desde el DomainSpec
- Operaciones CRUD automáticas por recurso
- Documentación OpenAPI accesible en `/docs`
- Código listo para levantar con `uvicorn`

```
@app.get("/users/{user_id}", response_model=User)
def get_user(user_id: int, db: Session =
    Depends(get_db)):
    return crud.get_user(db, user_id)
```



Insertar aquí screenshot del código generado o Swagger.

Limitaciones actuales y hoja de ruta

Limitaciones actuales

- Dependencia directa de la calidad del LLM
- Validación semántica aún débil
- Sin persistencia en base de datos real
- Sin agente corrector automático de errores

Trabajo futuro

- Validación mediante análisis AST del código
- Generación automática de tests
- Autenticación JWT y base de datos real
- Agente corrector con ciclos en LangGraph
- Interfaz web y contenedorización Docker

AgenteApis demuestra cómo una arquitectura multiagente puede automatizar la creación inicial de APIs backend usando IA, reduciendo fricción y acelerando el desarrollo.